

Bases de Datos No Relacionales (RA7)

Sistemas Distribuidos y Ecosistema NoSQL

Gestión de Bases de Datos

22 de abril de 2026

FP Madrid - DAM / DAW

1. Motivación: ¿Por qué NoSQL?

El contexto: La era del SQL (1970 - 2000)

Durante décadas, las bases de datos relacionales (MySQL, Oracle, SQL Server) fueron la única opción.

- **Foco:** Ahorrar espacio en disco (que era muy caro).
- **Técnica:** Normalización (dividir datos en muchas tablas para no repetir nada).
- **Garantía:** ACID (Consistencia absoluta).

¿Qué cambió?

La llegada de la Web 2.0 y el Big Data. El problema ya no es el espacio en disco, sino la **velocidad** y el **volumen** de los datos.

Caso 1: Redes Sociales (Complejidad)

El problema: El coste de los JOINS.

- En SQL, para saber qué le gusta a los amigos de “Juan”, el motor debe cruzar la tabla Usuarios, Amistades y Likes.
- Cada JOIN es una operación matemática costosa.
- **Recursividad:** “¿Qué recomiendan los amigos de mis amigos?”. En SQL esto requiere consultas anidadas que bloquean el servidor.

Rendimiento Exponencial

En SQL, cuanto más relación hay entre los datos, más lenta es la consulta. NoSQL (Grafos) soluciona esto con punteros físicos entre nodos.

Caso 2: Esquemas Rígidos y EAV

El problema: ¿Qué pasa si cada fila tiene columnas distintas? En SQL usamos el modelo **EAV (Entity-Attribute-Value)** para simular flexibilidad:

```
-- Para sacar un producto con 3 atributos:  
SELECT p.nombre, a1.valor as color, a2.valor as talla  
FROM Productos p  
JOIN Atributos a1 ON p.id = a1.p_id AND a1.attr = 'Color'  
JOIN Atributos a2 ON p.id = a2.p_id AND a2.attr = 'Talla';
```

- **Efecto:** El código se vuelve una “sopa de JOINS” ilegible.
- **NoSQL:** Simplemente guardamos un JSON con los campos que necesitamos para esa fila concreta.

Caso 3: Escrituras Masivas (IoT/Logs)

El problema: Los bloqueos (Locks).

- SQL es “celoso”. Para asegurar que nadie lea un dato mientras se escribe, bloquea filas o tablas enteras.
- Si tienes 10.000 sensores enviando datos por segundo, la base de datos pasa más tiempo gestionando bloqueos que escribiendo.

Escritura secuencial

Sistemas como Cassandra no “editan” datos, solo “añaden” al final de un archivo. Es como escribir en una libreta sin borrar nunca; es mucho más rápido.

Caso 4: Escalado Global (Latencia)

El problema: La física y la velocidad de la luz.

- Un paquete de red tarda unos 200ms en ir y volver de Madrid a Tokio.
- En SQL (ACID), una transacción no termina hasta que todos los nodos confirman.
- **Resultado:** Tu aplicación en Madrid va lenta porque el servidor de Tokio tarda en contestar.

Disponibilidad NoSQL

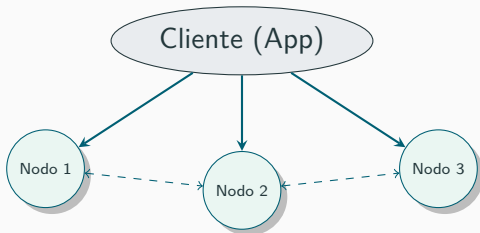
NoSQL permite que el servidor de Madrid conteste “OK” de inmediato y ya se sincronizará con Tokio unos milisegundos después.

2. Sistemas Distribuidos

2.1 ¿Qué es un Sistema Distribuido?

Definición simple: Es un grupo de ordenadores que trabajan juntos para que el usuario crea que está usando uno solo.

- **Nodo:** Cada servidor individual del sistema.
- **Cluster:** El conjunto de todos los nodos.
- **Red:** El “pegamento” que los une mediante mensajes (JSON, binario, etc.).



2.2 Escalabilidad: Vertical vs Horizontal

Vertical (Scaling UP)

- Comprar más RAM o CPU para el mismo servidor.
- **Problema:** Es carísimo y tiene un límite físico.

Horizontal (Scaling OUT)

- Añadir más servidores baratos al grupo.
- **Ventaja:** Es la base de NoSQL y de la nube (Cloud).

La analogía del transporte

Vertical es comprar un camión más grande. Horizontal es contratar a 10 mensajeros en moto.

2.3 Conceptos de Red: Latencia y Partición

Para entender NoSQL hay que entender qué falla en la red:

- **Latencia:** El tiempo que tarda un mensaje en viajar. Nunca es cero.
- **Partición de Red:** Cuando dos nodos no pueden hablar entre sí (se rompe el cable, falla el switch).

La importancia del fallo

En un sistema de un solo servidor (SQL clásico), si el servidor falla, todo para. En un sistema distribuido, si un nodo falla, el resto debe seguir trabajando.

2.4 Roles en el Cluster

No todos los nodos hacen lo mismo:

- **Primario / Maestro:** El que manda. Suele recibir las escrituras.
- **Secundario / Réplica:** Copia los datos del maestro. Sirve para lecturas rápidas o por si el maestro muere.
- **Árbitro:** Un nodo que no tiene datos, solo sirve para votar quién es el nuevo maestro si el actual falla.

2.5 Transparencia del Sistema

Un buen sistema distribuido debe ser transparente:

- **De ubicación:** Al programador le da igual en qué servidor esté el dato.
- **De fallo:** Si un nodo se desconecta, la aplicación ni se entera (el sistema redirige el tráfico).
- **De replicación:** El programador escribe una vez, el sistema lo copia a varios servidores.

Ejemplo práctico

En MongoDB, tú haces un `db.users.insert()`. No sabes si el dato se ha guardado en el servidor de Madrid o de París, ni cuántas copias se han hecho. El SGBD lo gestiona por ti.

3. El Agregado y el Modelado

3.1 ¿Qué es un Agregado? (RA7-CE.c)

Definición Profunda: Es un grupo de objetos relacionados que queremos tratar como una **unidad indivisible**.

- **SQL (La Despensa):** Tienes los huevos en un sitio, la harina en otro y la leche en la nevera. Para hacer una tarta (objeto), tienes que ir a buscarlos y mezclarlos (JOIN).
- **NoSQL (La Tarta ya hecha):** Guardas la tarta completa en una caja. Si quieres tarta, coges la caja y ya está. No pierdes tiempo buscando ingredientes.

La unidad de Atomicidad

En NoSQL, la base de datos te garantiza que o se guarda el “documento” entero o no se guarda nada. No hay términos medios.

3.2 Ejemplo Real: El Carrito de la Compra

¿Cómo vería un carrito una App de móvil?

- **En SQL:** Necesitarías la tabla CARRITOS, la tabla LINEAS_CARRITO y la tabla PRODUCTOS. Tres consultas o un triple JOIN.
- **En NoSQL:** Es un único “Agregado” JSON.

```
{  
  "cart_id": "ABC-123",  
  "items": [  
    { "id": "P01", "nombre": "Teclado", "precio": 85, "  
      tags": ["gaming", "rgb"] },  
    { "id": "P99", "nombre": "Cable", "precio": 12 }  
  ],  
  "total": 97,  
  "ultima_modificacion": "2024-04-16T10:00:00Z"  
}
```


3.3 Redundancia Controlada: El Snapshot

¿Por qué repetimos el nombre y el precio en el carrito si ya están en el catálogo?

1. **Velocidad (Performance):** Si el usuario abre el carrito 100 veces, le ahorramos a la base de datos 100 JOINS. Leer es casi instantáneo.
2. **Consistencia Histórica (Snapshot):** Si mañana el teclado sube de 85€ a 100€, el carrito que el usuario llenó hoy **debe** mantener los 85€. Si solo tuviéramos un ID al producto, el precio cambiaría “mágicamente” en el carrito del cliente.

La Redundancia es tu amiga

En NoSQL preferimos gastar un poco más de disco (que es barato) para comprar tiempo de CPU (que es caro) y coherencia en los datos.

4. CAP y PACELC

4.1 Teorema CAP: La dura realidad

Imagina que tienes dos servidores (A y B) sincronizados. De repente, se corta el cable entre ellos (**Partición**). Un usuario quiere escribir un dato en A. ¿Qué haces?

- **Opción 1: Consistencia (C).** Bloqueas el servidor A y dices “No puedo escribir hasta que vuelva la red”. El usuario se enfada porque el sistema no funciona, pero los datos son correctos.
- **Opción 2: Disponibilidad (A).** Dejas que el usuario escriba en A. El sistema funciona, pero el servidor B tiene datos viejos. Los datos no son iguales en todo el cluster.

La elección obligatoria

Como la red (**P**) fallará algún día, tienes que elegir: ¿Prefieres que tu sistema sea **CP** (siempre correcto pero a veces caído) o **AP** (siempre funcionando pero a veces con datos viejos)?

4.2 PACELC: ¿Y si la red NO falla?

El teorema CAP solo nos preocupa cuando hay problemas. El profesor Daniel Abadi dijo: “¿Y qué pasa el 99 % del tiempo cuando la red funciona bien?”.

- **PAC:** Si hay **P**artición, eliges entre **A**vailability o **C**onsistency.
- **ELC:** Else (si no hay fallo), eliges entre **L**atency o **C**onsistency.

El precio de la perfección

Incluso con la red perfecta, si quieres que el Nodo B tenga el dato *al mismo milisegundo* que el Nodo A, tienes que hacer esperar al usuario mientras el mensaje viaja. Esa espera es la **Latencia (L)**.

4.3 El modelo BASE: El hermano relajado de ACID

En SQL usamos ACID (Consistencia absoluta). En NoSQL usamos **BASE**:

- **BA (Basically Available):** El sistema parece que siempre funciona.
- **S (Soft state):** El estado de los datos puede cambiar con el tiempo sin que el usuario haga nada (porque los nodos se están sincronizando de fondo).
- **E (Eventual consistency):** Al final, si dejas de escribir, todos los nodos tendrán lo mismo. Pero “al final” puede ser dentro de 200ms.

5a. Bases de Datos de Clave-Valor

5.1 Concepto: El diccionario persistente

Es el modelo más sencillo y rápido que existe.

- **Estructura:** Un par Llave → Valor.
- **Opacidad:** Para la base de datos, el valor es una “caja negra” (BLOB). No sabe qué hay dentro, solo lo guarda y lo devuelve.
- **Analogía:** Es como las taquillas de un gimnasio. El número de taquilla es la **Llave** y lo que guardas dentro es el **Valor**. El gimnasio no sabe si guardas una mochila o unos zapatos.

SGBD Referente: Redis

Es ultra-rápido porque guarda los datos en la memoria RAM, no en el disco duro.

5.2 Arquitectura: ¿Cómo escala?

¿Cómo puede un cluster de 100 servidores saber dónde está una llave?

- **Hashing Consistente:** El sistema aplica una función matemática a la llave (Hash) para obtener un número.
- **El Anillo:** Ese número corresponde a un servidor específico en el cluster.

Velocidad $O(1)$

No importa si tienes 10 registros o 10 billones; el tiempo para encontrar una llave es siempre el mismo porque vas directo al servidor que la tiene.

5.3 Casos de Uso: ¿Cuándo usarlas?

No sirven para todo, pero en lo suyo son imbatibles:

- **Gestión de Sesiones:** Guardar si el usuario “Pepe” está logueado en la web.
- **Carritos de compra volátiles:** Datos que cambian mucho y se borran pronto.
- **Caché de base de datos:** Si una consulta SQL tarda 2 segundos, guardas el resultado en Redis y la próxima vez tardará 1 milisegundo.

Ejemplo real: Twitter

Usa Redis para guardar el 'Timeline' de los usuarios famosos y servirlo al instante a millones de personas.

5.4 Características: TTL y Tipos de Datos

Redis añade “superpoderes” al modelo básico:

- **TTL (Time To Live):** Puedes decirle: “Guarda esta sesión, pero bórrala automáticamente en 30 minutos”. Es ideal para autolimpieza.
- **No solo Strings:** El valor puede ser una **Lista** (para colas de tareas), un **Set** (para etiquetas únicas) o un **Hash** (para mini-objetos).

5.5 Ejemplo Práctico (Redis CLI)

Imagina que estamos en la terminal de Redis:

```
# Guardar un token de usuario que expira en 1 hora (3600 seg)
```

```
SET session:user:45 "token_a78fb2" EX 3600
```

```
# Recuperar el token
```

```
GET session:user:45
```

```
# Usar una lista para una cola de mensajes (mensajería)
```

```
LPUSH notificaciones:user:45 "Has recibido un mensaje"
```

```
LPUSH notificaciones:user:45 "Tu pedido ha salido"
```

```
# Sacar el último mensaje (LIFO)
```

```
LPOP notificaciones:user:45
```

5b. Bases de Datos Documentales

5.1 Concepto: El documento autodescriptivo

A diferencia de SQL, aquí no guardamos “filas”, guardamos **Documentos** (normalmente JSON o BSON).

- **Autodescriptivo:** Cada documento lleva sus propias “columnas” (llaves) dentro.
- **Jerárquico:** Puedes meter listas y otros objetos dentro de un documento.
- **SGBD Referente: MongoDB.**

La analogía del Archivador

SQL es una hoja de Excel rígida. NoSQL Documental es una carpeta con folios: cada folio puede tener una estructura distinta, pero todos están guardados en el mismo cajón.

5.2 Esquema Flexible (Schema-less)

¿Qué significa realmente que no hay esquema?

- **Polimorfismo:** En la colección alumnos, un alumno puede tener telefono y otro puede tener un array telefonos: ["casa", "movil"]. A la base de datos le da igual.
- **Evolución sin dolor:** Si tu App necesita un campo nuevo, lo añades en el código y listo. No hay que hacer un ALTER TABLE que bloquee la base de datos durante horas.

La responsabilidad es del programador

Como la base de datos no valida el esquema, es tu código el que debe asegurarse de que los datos tienen sentido.

5.3 Modelado: ¿Embeber o Referenciar?

Esta es la pregunta del millón en NoSQL Documental:

- **Embeber (Embedding):** Metes los datos hijos dentro del padre.
 - *Cuándo:* Relaciones 1:1 o 1:pocos (ej: direcciones de un usuario).
 - *Ventaja:* Lees todo de un golpe (Sin JOINS).
- **Referenciar (Linking):** Guardas el ID del hijo.
 - *Cuándo:* Relaciones 1:Muchos que crecen sin parar (ej: comentarios de un post viral).
 - *Ventaja:* Evitas que el documento supere el límite de tamaño (16MB en Mongo).

5.4 Indexación y Potencia de Consulta

Aunque sea NoSQL, puedes buscar de forma muy avanzada:

- **Índices secundarios:** Puedes crear un índice sobre el email, la edad o incluso sobre un campo que está dentro de una lista.
- **Agregaciones:** Puedes filtrar, agrupar y calcular medias como en SQL, pero usando un “pipeline” (tubería) de etapas.

Consultas Ad-hoc

A diferencia de Clave-Valor, aquí puedes decir: “Dame todos los usuarios de Madrid que tengan más de 20 años y les guste el Rock”.

5.5 Ejemplo Práctico (MongoDB Shell)

```
// Insertar un profesor con sus modulos embebidos
db.profesores.insertOne({
  nombre: "Vicente",
  especialidad: "Informatica",
  modulos: [
    { nombre: "Bases de Datos", horas: 6 },
    { nombre: "Sostenibilidad", horas: 3 }
  ],
  activo: true
})

// Buscar profesores que den mas de 5 horas de Bases de
  Datos
db.profesores.find({
  "modulos.nombre": "Bases de Datos",
  "modulos.horas": { $gt: 5 }
})
```

5c. Bases de Datos de Familias de Columnas

5.1 Concepto: El Mapa de Mapas

No pienses en tablas de Excel. Piensa en un **Mapa de Mapas** ordenado por una llave.

- **RowKey (Clave de Partición):** Es la llave que decide en qué servidor del cluster se guarda el dato físicamente.
- **Columna (Column):** Un par Nombre: Valor con marca de tiempo.
- **SGBD Referente: Apache Cassandra.**

Diferencia con SQL

En SQL, una fila es una unidad física. En Cassandra, la fila es solo una agrupación lógica de columnas bajo una misma clave.

5.2 ¿Cómo guardan los datos? (LSM-Trees)

¿Por qué Cassandra escribe más rápido que cualquier SQL?

- **Escritura Secuencial:** No “busca” el dato en el disco para editarlo (eso es lento). Simplemente escribe la nueva versión al final de un archivo de log.
- **Memtable:** Los datos se guardan en RAM y se vuelcan a disco en bloque.

No hay borrados reales

Si borras un dato, se escribe una marca llamada “Tombstone” (lápida). El dato se borrará físicamente más tarde durante una limpieza automática (Compaction).

5.3 Tablas Dispersas (Sparse Tables)

En SQL, si una tabla tiene 100 columnas y solo rellenas 2, el resto son NULLs que ocupan sitio o lógica.

- **Sin NULLs:** En una BBDD de columnas, si una fila no tiene el dato “Email”, la columna “Email” simplemente **no existe** para esa fila. No ocupa ni un bit.
- **Familias de Columnas:** Agrupamos columnas que suelen leerse juntas (ej: “Datos Personales” separada de “Estadísticas de Uso”).

5.4 Modelado: Query-First (Diseño por consulta)

¡Aquí la normalización está prohibida!

- **Primero la pregunta, luego la tabla:** No diseñas tablas para guardar datos, diseñas tablas para responder preguntas.
- **Desnormalización total:** Si necesitas ver los usuarios por Ciudad y también por Edad, creas **dos tablas** con los mismos datos repetidos, optimizadas para cada búsqueda.

La regla de oro

Un SELECT nunca debe requerir un JOIN. Si necesitas datos de dos sitios, guárdalos ya juntos.

5.5 Ejemplo Visual de Almacenamiento

Imagina una tabla de usuarios en Cassandra:

RowKey (ID)	Family: Perfil	Family: Actividad
user_1	{nombre: "Ana", edad: 20}	{login: "2024-04-16"}
user_2	{nombre: "Luis"}	{login: "2024-04-15", ip: "10.0.0.1"}

- **Dato Clave:** Luis no tiene campo 'edad'. Luis tiene un campo 'ip' que Ana no tiene.
- **Físico:** El disco guarda la familia "Perfil" en un archivo y "Actividad" en otro. Si solo quieres nombres, el disco no toca los datos de actividad.

5d. Bases de Datos de Grafos

5.1 Concepto: La relación es lo primero

En todas las BBDD anteriores, la relación era un ID guardado en algún sitio. Aquí la relación es un objeto con nombre y propiedades.

- **Nodos (Vértices):** Las entidades (ej: Persona, Ciudad, Producto).
- **Relaciones (Aristas):** Las conexiones (ej: CONOCE_A, VIVE_EN, COMPRÓ).
- **Propiedades:** Atributos de ambos (ej: Maria tiene 20 años, Maria conoce a Juan desde 2010).
- **SGBD Referente: Neo4j.**

5.2 Adyacencia sin índices (Index-free Adjacency)

¿Por qué son tan rápidos buscando relaciones?

- **Punteros físicos:** Cada nodo tiene grabada la dirección de memoria de sus vecinos.
- **Sin búsquedas:** Para ir de “Juan” a sus “Amigos”, el disco no busca en una tabla; simplemente salta a las direcciones de memoria que ya conoce.

La analogía del laberinto

SQL es como un laberinto donde en cada esquina tienes que mirar un mapa (Índice). Grafos es como un laberinto donde hay un hilo de Ariadna que te lleva directo a la salida.

5.3 ¿Cuándo usarlas? (Conectividad)

Si tu pregunta empieza por “¿Cómo están conectados...?”, usa Grafos.

- **Redes Sociales:** Amigos de amigos, gente que quizás conozcas.
- **Detección de Fraude:** ¿Hay un grupo de personas enviándose dinero en círculo para blanquearlo?
- **Recomendaciones:** “Gente que compró este libro y vive en tu ciudad también compró este otro”.
- **Logística:** Calcular la ruta más rápida entre dos puntos.

5.4 Lenguaje de Consulta: Cypher

Es el lenguaje de Neo4j. Es muy intuitivo porque usa “dibujos” hechos con texto (ASCII Art).

- Los nodos van entre paréntesis: (p:Persona)
- Las relaciones van entre corchetes con flechas: -[:CONOCE]->
- Los filtros son declarativos, como en SQL.

Ejemplo de patrón

```
(juan)-[:AMIGO_DE]->(maria)
```

5.5 Ejemplo Práctico (Cypher)

Imagina que queremos buscar recomendaciones de amigos:

```
// Crear Maria y Juan y su relacion
CREATE (p1:Persona {nombre: "Maria"})
CREATE (p2:Persona {nombre: "Juan"})
CREATE (p1)-[:CONOCE {desde: 2020}]->(p2)

// Buscar "Amigos de mis amigos" que no sean mis amigos directos
MATCH (yo:Persona {nombre: "Juan"})-[:AMIGO_DE]->(amigo)
    -[:AMIGO_DE]->(fof)
WHERE NOT (yo)-[:AMIGO_DE]->(fof)
RETURN fof.nombre
```

6. Conclusión y Futuro

6.1 Persistencia Políglota: El mundo real

¿Cuál es la mejor base de datos? **Ninguna y todas a la vez.**

- En una empresa moderna, se usa la base de datos adecuada para cada parte de la App.
- **SQL:** Para la pasarela de pagos y transacciones bancarias (ACID).
- **Redis:** Para las sesiones de usuario y caché de la web.
- **MongoDB:** Para el catálogo de productos y el perfil del usuario.
- **Neo4j:** Para el sistema de recomendaciones (“A otros usuarios también les gustó...”).

No seas fanático

Un buen arquitecto de software no se casa con una tecnología. Elige la herramienta que mejor resuelva el problema del cliente.

6.2 Matriz de Selección (RA7-CE.b)

Tipo	¿Cuándo usarla?	Punto Fuerte	Escalado
Relacional	Datos estructurados	Integridad (ACID)	Vertical
Clave-Valor	Caché, Sesiones	Latencia mínima	Horizontal
Documental	Apps Web, Catálogos	Flexibilidad	Horizontal
Columnares	Big Data, Logs	Escritura masiva	Horizontal
Grafos	Relaciones complejas	Conectividad	Complejo

6.3 Resumen del RA7: ¿Qué hemos aprendido?

Al finalizar este tema, debes ser capaz de:

1. **Diferenciar** los modelos NoSQL del modelo relacional.
2. **Identificar** qué tipo de base de datos conviene según el caso de uso.
3. **Entender** el impacto de la red y la distribución (Teorema CAP).
4. **Modelar** datos usando agregados y redundancia controlada.

Próximo paso: La Práctica

Instalaremos un cluster de MongoDB y aprenderemos a realizar consultas complejas y pipelines de agregación.

“Las bases de datos relacionales te dicen cómo deben ser tus datos. Las bases de datos NoSQL te preguntan qué quieres hacer con ellos.”

¿Preguntas?